

Day2

1. Data Type: `wchar_t`

Type	Size	Range	Use Case
<code>char</code>	1 byte	0 – 255 (256 values)	ASCII characters
<code>wchar_t</code>	2 bytes	0 – 65,535 (65,536 values)	Wide characters (Unicode, multilingual support)

```
char    c = 'A';           // ASCII
wchar_t w = L'अ';          // Wide character literal – note the L prefix
```

`wchar_t` is useful when working with non-English scripts (Hindi, Chinese, etc.) or Windows API calls that require Unicode strings.

2. Structure in C vs C++

Structure in C

A structure is a collection of **similar and dissimilar data types** grouped under one name.

```
struct Student {
    int    rollno;
    double marks;
};
```

Limitations in C:

- Everything is **global by default** — no access control.
- **Functions cannot be written** inside a structure.
- No data security — all members are publicly accessible.

Structure in C++

C++ extends structures with two major enhancements:

1. **Access Specifiers** — control visibility of members (data security).
2. **Member Functions** — functions can be written inside a structure, achieving **encapsulation** (wrapping data and code together).

```
struct Student {  
private:  
    int    rollno;  
    double marks;  
  
public:  
    void setData(int r, double m) {  
        rollno = r;  
        marks  = m;  
    }  
  
    void display() {  
        cout << "Roll No: " << rollno << ", Marks: " << marks << endl;  
    }  
};
```

In C++, `struct` members are **public by default**. This is the only difference between `struct` and `class` in C++ — `class` members are **private by default**.

3. Access Specifiers

Used to **control the visibility** of data members and member functions.

Specifier	Accessible From
<code>private</code>	Only within the same class/struct
<code>public</code>	Anywhere (inside or outside the class)
<code>protected</code>	Within the class and its derived classes (<i>covered later — inheritance</i>)

```
class Box {  
private:  
    int length;    // hidden from outside  
  
public:  
    void setLength(int l) { length = l; } // controlled access  
    int  getLength()      { return length; }  
};
```

4. Namespace

A **namespace** groups logically related identifiers (variables, functions, classes) to avoid **name conflicts**.

Language	Equivalent Concept
Java	<code>package</code>

Language	Equivalent Concept
.NET/C#	namespace
C++	namespace

```
namespace MathUtils {
    int add(int a, int b) { return a + b; }
}

namespace StringUtils {
    int add(int a, int b) { return a + b; } // no conflict
}

// Usage
MathUtils::add(3, 4);
StringUtils::add(3, 4);

// Or bring into scope
using namespace MathUtils;
add(3, 4);
```

`std` in `std::cout` is itself a namespace — the **Standard Library namespace**.

5. `cin` and `cout`

- Defined in `<iostream>`, part of the `std` namespace.
- Replace `scanf/printf` from C.

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter age: "; // output (insertion operator <<)
    cin >> age;             // input (extraction operator >>)
    cout << "Age: " << age << endl;
    return 0;
}
```

C Style	C++ Style
<code>printf()</code>	<code>cout <<</code>
<code>scanf()</code>	<code>cin >></code>